

PROMPT ENGINEERING

Principles, Techniques,
and Applications



Prompt Engineering	3
1. Prompting Techniques	4
1.1 Zero-Shot Prompting	4
1.2 One-Shot Prompting	4
1.3 Few-Shot Prompting	5
1.4 Chain-of-Thought (CoT) Prompting	6
1.5 Role Prompting	7
1.6 Instruction Prompting	8
2. Implementation of Simple Chatbot	9

Prompt Engineering

Prompt engineering is the technique used to design prompts that helps us to get the best possible output from LLMs. It is the primary way non-engineers interact and control LLM behavior. A well-designed prompt can transform the mediocre response to expert-level responses.

The same model with the poor prompt gives a mediocre answer. With an excellent prompt, it gives the expert-level answer. Prompt Engineering is one of the most cost effective skills that anyone working with LLMs can have as it costs nothing (no GPU, no fine-tuning) and can dramatically improve the quality.

There are several prompt engineering techniques. We will discuss then one by one in this session:

1. Zero-Shot Prompting
2. One-Shot Prompting
3. Few-Shot Prompting
4. Chain-of-Thought (CoT) Prompting
5. Role Prompting
6. Instruction Prompting

1. Prompting Techniques

1.1 Zero-Shot Prompting

In **Zero-Shot Prompting**, a model is given only a task without any examples. The model depends entirely on its pre-trained knowledge to generate a response.

```
zero_shot_prompt = """
Classify the sentiment of these reviews as:
"Positive", "Negative", "Neutral"
Here is the sentence:
"The hotel was okay. Room was clean but service was slow."
"""

response = model.invoke(zero_shot_prompt)
print(response.content)
```

Neutral

Zero-Shot Prompting works well for simple and common tasks (sentiment, translation, summarization). It also works well when the task is unambiguous and straight-forward.

1.2 One-Shot Prompting

In **One-Shot Prompting**, a model is given only a task with one example.

```
one_shot_prompt = """
Example:
English: Hello
Nepali: Namaste
Now translate:
English: Good Morning
"""
```

```
from IPython.display import Markdown
response = model.invoke(one_shot_prompt)
display(Markdown(response.content))
```

English: Good Morning
Nepali: शुभ प्रभात (shubh prabhat)

One-Shot Prompting helps to establish the expected format and improves consistency.

1.3 Few-Shot Prompting

In **Few-Shot Prompting**, a model is provided with 2-5 examples of input → output pairs before actual query. This is an extremely powerful technique because it provides the exact format, tone, and style to the model, without any training or fine-tuning.

```
few_shot_prompt = """
Convert each sentence into a suitable emoji. Here are some examples:
Sentence: I got 4 GPA in high school exams.
Emoji: 🎉📚
Sentence: It's raining outside.
Emoji: 🌧️☔
Sentence: I am in love with a girl.
Emoji: 💕👧
Sentence: I am going to propose to a girl for marriage.
Emoji:
"""
```

```
response = model.invoke(few_shot_prompt)
print(response.content)
```

Emoji: 💍👧

1.4 Chain-of-Thought (CoT) Prompting

In **Chain-of-Thought Prompting**, a model is prompted to work step by step before giving a final answer. This improves performance of models on tasks that require multi-step reasoning, math and logic.

```
# without CoT Prompting:
no_cot_prompt = """
If 6 printers can print 6 pages in 6 minutes,
how many pages can 12 printers print in 12 minutes?
"""

response = model.invoke(no_cot_prompt)
display(Markdown(response.content))
```

12 printers can print 12 * 1 page in 12 minutes. **#WRONG**

```
# with CoT Prompting:
cot_prompt = """
If 6 printers can print 6 pages in 6 minutes,
how many pages can 12 printers print in 12 minutes?

Solve the problem step by step before giving the final answer.
"""

response = model.invoke(cot_prompt)
display(Markdown(response.content))
```

To solve this problem, let's follow the given information step by step.

1. 6 printers can print 6 pages in 6 minutes. This means that 1 printer can print 1 page in 6 minutes. (Because 6 printers print 6 pages in 6 minutes, it's the same as 1 printer printing 1 page in 6 minutes)

So, we can say that the work rate of 1 printer is 1 page per 6 minutes.

.....

.....

So, 12 printers can print **24 pages** in 12 minutes

1.5 Role Prompting

In **Role Prompting**, the model is assigned a specific role (persona). Assigning a specific role (persona) dramatically changes the depth, vocabulary and approach of the response.

```
# Without role prompting
prompt = "Explain playing cards in short."
response = model.invoke(prompt)
display(Markdown(response.content))
```

Here's a brief overview of a standard playing card:

Components:

Deck: A deck consists of 52 cards.

Suits: There are 4 suits: Hearts, Diamonds, Clubs, and Spades.

.....

Card Values:

Face cards: Jack, Queen, and King are worth 10 points each.

Number cards: Cards 2-10 are worth their face value.

Ace: Can be worth either 1 or 11 points, depending on the game.

```
# With role prompting
prompt = """
You are a math Professor who excels in probability.
Explain playing cards in short.
"""
response = model.invoke(prompt)
display(Markdown(response.content))
```

Welcome to the world of probability!

The Deck:

A standard deck of playing cards contains 52 cards, without jokers. It consists of:

- **4 suits:** Hearts (H), Diamonds (D), Clubs (C), and Spades (S)
- **13 ranks within each suit:** Ace (A), 2, 3, 4, 5, 6, 7, 8, 9, 10, Jack (J), Queen (Q), King (K)

.....

Key Probabilities:

1. **Total possible combinations:** 52
2. **Suit probability:** Each suit (H, D, C, S) has 25% chance of being drawn

As we can see, assigning a specific role significantly changes the model's response. Without role prompting, the explanation is generic and focuses only on basic components of playing cards. But when the model is assigned the role of math professor, it explains the same topic from a probability perspective.

1.6 Instruction Prompting

In **Role Prompting**, the model is provided with clear, specific instructions to produce better and more controlled outputs than vague prompts. Here is an example of instruction prompting:

```
instruction_prompting = """
Summarize the following article.
Instructions:
- Use exactly 4 bullet points.
- Keep each bullet under 15 words.
- Do not include opinions.
- Focus only on the key facts.
- Please write it in proper markdown

{article}
"""
```

```
# reading the file for {article}
with open("Documents/article.txt", "r", encoding="utf-8") as f:
    article = f.read()
print(article)
```

```
# sending the prompt with article to LLM
response = model.invoke(instruction_prompting.format(article=article))
display(Markdown(response.content))
```

Key Facts about Prabhas

1. Prabhas is a Telugu actor born on October 23, 1979.
2. He has over 24 film appearances and seven Filmfare nominations
3. Prabhas is the first pan-Indian star and highest-paid Indian actor.
4. He holds multiple records in Indian cinema, including a film grossing over ₹1000 crore worldwide.

2. Implementation of Simple Chatbot